

Apache Hive

Introduction to Hive

1. Big Data Query Problem

Before Hive, Hadoop users had a major challenge.

Imagine a dataset stored in HDFS.

Example log dataset:

```
user_id,page,timestamp,country
101,home,10:00,USA
101,product,10:02,USA
102,home,10:05,UK
103,checkout,10:10,USA
```

Business questions:

- How many users visited each page?
- Which country generates the most traffic?
- What pages are most popular?

Without Hive, this required writing **Java MapReduce programs**.

MapReduce workflow:

Mapper → parse data

Reducer → aggregate

Driver → configure job

Jar → submit to cluster

Even a simple query could require **200–300 lines of Java code**.

Hive was created to solve this problem.

2. What Hive Is

Hive is a **data warehouse system on top of Hadoop**.

It allows users to query large datasets using **SQL-like language (HiveQL)**.

Example query:

```
SELECT country, COUNT(*)  
FROM web_logs  
GROUP BY country;
```

Hive internally converts this query into:

MapReduce Job

or

Tez Job

or

Spark Job

The user writes SQL, and Hive handles distributed execution.

3. Where Hive Fits in Data Engineering

Typical pipeline:

Raw Logs

↓

Kafka / NiFi

↓

HDFS / S3

↓

Hive Tables

↓
Analytics Queries
↓
Dashboard / ML / Reports

Hive is primarily used for:

- Batch analytics
- Data warehousing
- SQL queries on big data

4. First Lab — Connect to Hive

Start Docker stack

```
docker compose up -d
```

Check services

```
docker ps
```

You should see containers like:

```
namenode  
datanode  
hive-server  
hive-metastore  
postgres  
spark-master
```

Enter Hive Container

```
docker exec -it hive-server bash      docker exec -it hive-server /opt/hive/bin/beeline -u  
jdbc:hive2://localhost:10000 -n hive
```

Start Hive CLI

```
beeline -u jdbc:hive2://localhost:10000
```

If successful:

Connected to Apache Hive

5. Working With Databases

Hive organizes data using **databases and tables**.

Check databases:

```
SHOW DATABASES;
```

Typical result:

```
default
```

Create a database:

```
CREATE DATABASE training;
```

Switch database:

```
USE training;
```

Verify:

```
SHOW TABLES;
```

Currently empty.

Exercise

- What database are you using?
- How many databases exist?
- What command switches databases?

```
SET hive.execution.engine=tez;
```

Hour 2 — Hive Tables and Data Loading

6. Hive Tables

Hive tables are **metadata definitions for data stored in HDFS**.

Example file in HDFS:

```
/data/employees.csv
```

Hive table defines structure.

Example schema:

```
id INT
name STRING
department STRING
salary DOUBLE
```

Hive does **not store the data itself**, it references the file.

7. Upload Data to HDFS (Lab)

Open namenode container:

```
docker exec -it namenode bash
```

Create directory:

```
hdfs dfs -mkdir /data
```

Create dataset:

```
nano employees.csv
```

Insert:

```
1,John,Engineering,90000
2,Alice,HR,70000
3,Bob,Engineering,85000
4,Sara,Finance,95000
5,David,Engineering,88000
```

```
cat > employees.csv <<EOF
1,John,HR,50000
2,Alice,IT,65000
3,Bob,Finance,70000
4,Meena,IT,72000
EOF
```

Upload file:

```
hdfs dfs -put employees.csv /data/
```

Verify:

```
hdfs dfs -ls /data
```

Expected:

```
/data/employees.csv
```

8. Create Hive Table (Lab)

Return to Hive.

Create table:

```
CREATE TABLE employees(
  id INT,
  name STRING,
  department STRING,
  salary DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

Check table schema:

```
DESCRIBE employees;
```

9. Load Data Into Table

Load data from HDFS.

```
LOAD DATA INPATH '/data/employees.csv'  
INTO TABLE employees;
```

Verify:

```
SELECT * FROM employees;
```

Result:

```
1 John Engineering 90000  
2 Alice HR 70000  
3 Bob Engineering 85000
```

Exercise

Write queries:

- 1 Show all employees
- 2 Show only Engineering employees
- 3 Show salary greater than 85000

Querying Data in Hive

10. Filtering Data

Basic syntax:

```
SELECT columns  
FROM table  
WHERE condition;
```

Example:

```
SELECT name, salary
FROM employees
WHERE department='Engineering';
```

Operators supported:

```
=
>
<
BETWEEN
LIKE
IN
```

Example:

```
SELECT *
FROM employees
WHERE salary > 85000;
```

11. Aggregations

Aggregation functions summarize data.

Common functions:

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

Example:

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

Result:

Engineering 87666
HR 70000
Finance 95000

Lab — Aggregation Queries

Count employees

```
SELECT COUNT(*) FROM employees;
```

Department totals

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department;
```

Highest salary

```
SELECT MAX(salary)  
FROM employees;
```

12. Sorting Data

Global sort:

```
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

Hive sorts the entire dataset.

13. Joins

Create departments table.

```
CREATE TABLE departments(  
  id INT,  
  dept STRING
```

```
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ',';
```

Insert values:

```
INSERT INTO departments VALUES  
(1, 'Engineering'),  
(2, 'HR'),  
(3, 'Finance');
```

Join example:

```
SELECT e.name, d.dept  
FROM employees e  
JOIN departments d  
ON e.department = d.dept;
```

Exercise

Write queries:

- Join employees and departments
- Show employee name and department

Performance Optimization

14. Partitioning

Partitioning divides large datasets into folders.

Example dataset:

sales data for multiple years

Without partition:

```
/sales_data/all_data.csv
```

With partition:

```
/sales_data/year=2023
```

```
/sales_data/year=2024
```

This allows Hive to skip scanning unnecessary data.

Lab — Partition Table

Create table:

```
CREATE TABLE sales(  
  id INT,  
  product STRING,  
  amount DOUBLE  
)  
PARTITIONED BY (year INT)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ',';
```

Create data files in namenode.

```
sales2023.csv
```

```
1,Laptop,1000
```

```
2,Phone,600
```

```
3,Tablet,400
```

```
sales2024.csv
```

```
4,Laptop,1200
```

```
5,Phone,700
```

```
6,Tablet,450
```

```
cat > SALES2023.csv <<EOF
```

```
1,Laptop,1000 2,Phone,600 3,Tablet,400  
EOF
```

```
cat > SALES2024.csv <<EOF
```

```
4,Laptop,1200 5,Phone,700 6,Tablet,450  
EOF
```

Upload to HDFS.

```
hdfs dfs -put sales2023.csv /data/
```

```
hdfs dfs -put sales2024.csv /data/
```

Load partitions:

```
LOAD DATA INPATH '/data/sales2023.csv'  
INTO TABLE sales  
PARTITION(year=2023);  
LOAD DATA INPATH '/data/sales2024.csv'  
INTO TABLE sales  
PARTITION(year=2024);
```

Query:

```
SELECT * FROM sales WHERE year=2024;
```

Explain concept:

Hive scans only

/year=2024/

This is **partition pruning**.

15. File Formats

Hive supports multiple storage formats.

Format	Type	Use
TEXTFILE	Row	default
PARQUET	Column	analytics
ORC	Column	Hive optimized

Column formats read only required columns.

Lab — Parquet Table

Create table:

```
CREATE TABLE employees_parquet(  
  id INT,
```

```
name STRING,  
department STRING,  
salary DOUBLE  
)  
STORED AS PARQUET;
```

Insert data:

```
INSERT INTO employees_parquet  
SELECT * FROM employees;
```

Query:

```
SELECT AVG(salary)  
FROM employees_parquet;
```

Explain performance benefits

Final Mini Project (Hands-on)

Dataset:

```
orders.csv  
order_id,user_id,product,amount,country,year  
1,101,phone,700,USA,2023  
2,102,laptop,1200,UK,2024  
3,101,tablet,500,USA,2024
```

- 1** Upload file to HDFS
- 2** Create Hive table
- 3** Partition by year
- 4** Load data
- 5** Run analytics queries

Example query:

```
SELECT country, SUM(amount)  
FROM orders  
GROUP BY country  
ORDER BY SUM(amount) DESC;
```

End-to-End Hive Data Engineering Pipeline Lab

Dataset: **E-commerce Orders**

Pipeline Architecture

```
CSV Source
  ↓
HDFS (/data/orders/)
  ↓
Hive RAW Table
  ↓
Hive CLEAN Table
  ↓
Hive ANALYTICS Table
  ↓
Business Insights
```

Each layer has a purpose.

Layer	Purpose
RAW	store original data
CLEAN	apply transformations
ANALYTICS	aggregated metrics

This is similar to **Bronze → Silver → Gold** architecture.

Step 1 — Create Source Dataset

Enter the NameNode container:

```
docker exec -it namenode bash
```

Create dataset:

```
nano orders.csv
```

Insert data:

```
order_id,user_id,product,category,amount,country,order_date
1,101,Phone,Electronics,700,USA,2024-01-01
2,102,Laptop,Electronics,1200,UK,2024-01-01
3,103,Tablet,Electronics,500,USA,2024-01-02
4,104,Shoes,Fashion,200,India,2024-01-02
5,105,Jacket,Fashion,300,USA,2024-01-03
6,106,Headphones,Electronics,150,UK,2024-01-03
7,107,Watch,Fashion,400,USA,2024-01-04
```

Step 2 — Upload Data to HDFS

Create directory:

```
hdfs dfs -mkdir -p /data/orders
```

Upload file:

```
hdfs dfs -put orders.csv /data/orders/
```

Verify:

```
hdfs dfs -ls /data/orders
```

Expected:

```
/data/orders/orders.csv
```

HDFS is the storage layer of the data lake.

Step 3 — Connect to Hive

Enter Hive container:

```
docker exec -it hive-server bash
```

Connect using Beeline:

```
beeline -u jdbc:hive2://localhost:10000/default
```

Step 4 — Create Database

```
CREATE DATABASE ecommerce;
```

Switch database:

```
USE ecommerce;
```

Verify:

```
SHOW TABLES;
```

Step 5 — Create RAW Layer Table

Explain concept:

RAW tables should usually be **EXTERNAL tables**.

Reason:

- data lives in HDFS
- multiple tools can access it
- Hive should not delete the data

Create table:

```
CREATE EXTERNAL TABLE orders_raw(  
  order_id INT,  
  user_id INT,  
  product STRING,  
  category STRING,  
  amount DOUBLE,  
  country STRING,  
  order_date STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION '/data/orders'  
TBLPROPERTIES ("skip.header.line.count"="1");
```

Explain:

Component	Purpose
EXTERNAL	Hive does not control data
LOCATION	points to HDFS folder
skip.header	ignores CSV header

Step 6 — Query RAW Data

Run:

```
SELECT * FROM orders_raw;
```

Explain:

RAW layer is **not optimized or cleaned**.

Step 7 — Create CLEAN Layer

Explain purpose:

CLEAN layer performs:

- data validation
- type casting
- formatting
- partitioning

Create table:

```
CREATE TABLE orders_clean(  
  order_id INT,  
  user_id INT,  
  product STRING,  
  category STRING,  
  amount DOUBLE,  
  country STRING,  
  order_date DATE  
)  
STORED AS PARQUET;
```

Explain:

Why Parquet?

- columnar format
- compression
- faster queries

Step 8 — Transform Data

Insert cleaned data:

```
INSERT INTO orders_clean  
SELECT  
  order_id,  
  user_id,
```

```
product,  
category,  
amount,  
country,  
CAST(order_date AS DATE)  
FROM orders_raw;
```

Explain transformation:

Operation	Purpose
CAST	convert string to date
PARQUET	efficient storage

Step 9 — Verify Clean Table

```
SELECT * FROM orders_clean;
```

Explain:

CLEAN layer is **analytics ready**.

Step 10 — Create Analytics Table

Analytics layer stores **aggregated metrics**.

Example business question:

What is the total revenue by country?

Create table:

```
CREATE TABLE revenue_by_country  
STORED AS PARQUET
```

```
AS
SELECT
  country,
  SUM(amount) AS total_revenue,
  COUNT(*) AS total_orders
FROM orders_clean
GROUP BY country;
```

Explain:

This is **pre-computed analytics**.

Step 11 — Run Business Queries

Query revenue:

```
SELECT * FROM revenue_by_country;
```

Expected output:

country	revenue
USA	1900
UK	1350
India	200

Top selling category:

```
SELECT
  category,
  SUM(amount) revenue
FROM orders_clean
GROUP BY category
ORDER BY revenue DESC;
```

Daily sales:

```
SELECT
  order_date,
  SUM(amount)
FROM orders_clean
GROUP BY order_date;
```

Step 12 — Explain Real Production Workflow

Explain how this mirrors real pipelines.

Example architecture:

```
Kafka → HDFS
      ↓
Hive RAW tables
      ↓
Hive CLEAN tables
      ↓
Analytics tables
      ↓
Tableau / Power BI
```

Final Exercise

1 Category analytics table

revenue_by_category

Expected query:

```
CREATE TABLE revenue_by_category AS
SELECT
  category,
  SUM(amount) revenue
```

```
FROM orders_clean  
GROUP BY category;
```

2 Top 3 customers

```
SELECT  
  user_id,  
  SUM(amount) total_spent  
FROM orders_clean  
GROUP BY user_id  
ORDER BY total_spent DESC  
LIMIT 3;
```

```
SET hive.exec.orc.default.compress=SNAPPY;
```

Benefits:

- smaller storage
- less disk I/O
- faster queries

Optimization 4 — Indexing

ORC automatically builds **lightweight indexes**.

Indexes store:

- row positions
- column statistics
- min/max values

Hive can jump directly to the relevant part of a file.

Optimization 5 — Vectorized Execution

Hive can process ORC data in **batches instead of row-by-row**.

Traditional execution:

```
process row  
process row  
process row
```

Vectorized execution:

```
process 1024 rows at once
```

Enable vectorization:

```
SET hive.vectorized.execution.enabled=true;
```

This can make queries **5–10x faster**.

Optimization 6 — Bloom Filters

ORC supports **bloom filters** for fast lookups.

Example:

```
SELECT * FROM users  
WHERE user_id = 101;
```

Bloom filter quickly determines whether value may exist in a stripe.

This avoids scanning unnecessary data.

5. ORC vs Other Formats

Format	Type	Use Case
TEXTFILE	row	simple storage
AVRO	row	streaming data
PARQUET	column	general analytics
ORC	column	Hive optimized

Key difference:

ORC → optimized specifically for Hive engine

PARQUET → optimized for Spark + general analytics

6. ORC in Hive Pipeline (Lab)

Pipeline so far:

HDFS → orders_raw → orders_clean

Now we add:

orders_orc

Step 1 — Create ORC Table

Inside Hive:

```
CREATE TABLE orders_orc(  
  order_id INT,  
  user_id INT,  
  product STRING,  
  category STRING,  
  amount DOUBLE,  
  country STRING,
```

```
    order_date DATE
)
STORED AS ORC;
```

Step 2 — Insert Data

```
INSERT INTO orders_orc
SELECT * FROM orders_clean;
```

Hive converts the data into ORC format.

Step 3 — Compare Query Plans

Run:

```
EXPLAIN
SELECT SUM(amount)
FROM orders_clean;
```

Then:

```
EXPLAIN
SELECT SUM(amount)
FROM orders_orc;
```

Explain how ORC allows:

```
column pruning
predicate pushdown
vectorized execution
```

Step 4 — Check File Size

Inside NameNode container:

```
hdfs dfs -du -h /user/hive/warehouse/ecommerce.db/
orders_clean (PARQUET)
orders_orc (ORC)
```

ORC often uses **less storage**.

ORC VS PARQUET

1. History: Why ORC and Parquet Were Created

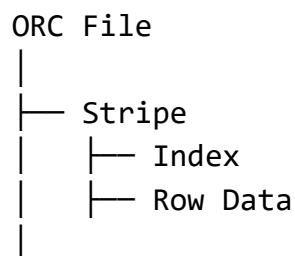
Understanding this makes everything clear.

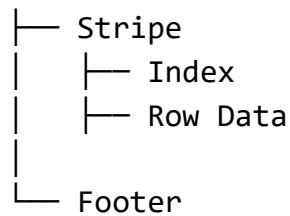
Format	Created by	Target engine
ORC	Hive team at Hortonworks	Hive
Parquet	Twitter + Cloudera	Spark / Impala / Presto

So the design goals were different.

2. ORC Architecture (Designed for Hive Execution Model)

ORC files are structured using **large stripes** optimized for Hive's execution engine.





Typical stripe size:

64 MB – 256 MB

Why this matters:

Hive historically used **MapReduce**.

MapReduce reads **large sequential blocks of data**.

Large stripes improve:

sequential disk I/O

less task overhead

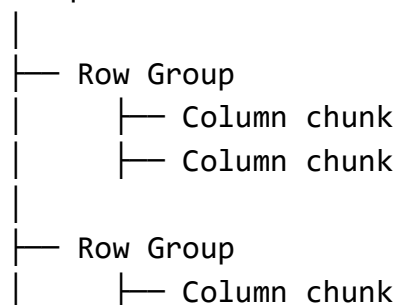
better compression

So ORC was optimized for **large batch scans**.

3. Parquet Architecture (Designed for Distributed Query Engines)

Parquet organizes files using **row groups and column chunks**.

Parquet File



|
└─ Metadata

Typical row group size:

8 MB – 128 MB

Why this matters:

Spark and distributed engines operate on **parallel column reads across partitions**.

Parquet allows:

fine-grained column access
better parallelism
smaller task execution units

4. Metadata Structure Differences

This is one of the **biggest practical differences**.

ORC metadata

Stored per:

stripe
row group
column

Includes:

min value
max value
count
bloom filters
row indexes

Hive can skip stripes quickly.

Example query:

```
SELECT * FROM orders WHERE amount > 1000;
```

Hive checks stripe metadata:

Stripe1 amount 10-500 → skip

Stripe2 amount 800-2000 → scan

This is extremely efficient for Hive.

Parquet metadata

Stored per:

row group

column chunk

Includes:

min/max statistics

dictionary encoding

column indexes

Spark uses this metadata to:

parallelize column scans

perform predicate pushdown

optimize distributed execution

5. Compression Strategy

Both support compression, but ORC has **Hive-optimized compression pipelines**.

ORC compression

Column-specific encoding:

dictionary encoding

run length encoding

delta encoding
bit packing

Combined with:

Snappy
Zlib
LZO

Result:

very high compression
very fast scans

Hive queries often run **10–15% faster on ORC than Parquet.**

Parquet compression

Parquet focuses on:

dictionary encoding
run-length encoding
bit packing

But compression strategy is designed for **cross-engine compatibility.**

That's why Parquet is used by:

Spark
Presto
Trino
Athena
BigQuery
Snowflake

6. Execution Engine Integration

This is where the biggest difference appears.

Hive + ORC

Hive supports **vectorized execution optimized specifically for ORC**.

Example:

Row processing → slow

Batch processing → fast

Hive processes:

1024 rows at once

But vectorization support is **strongest for ORC**.

Enable:

```
SET hive.vectorized.execution.enabled=true;
```

Spark + Parquet

Spark's Catalyst optimizer and Tungsten engine are heavily tuned for Parquet.

Spark benefits include:

column pruning

predicate pushdown

dictionary filtering

memory-efficient scanning

Spark reading Parquet is extremely efficient because:

Spark DataFrame API was designed around Parquet structure

7. Ecosystem Compatibility

This is another practical reason.

Format

Ecosystem Support

ORC	Hive, Presto
Parquet	Spark, Trino, Athena, BigQuery, Snowflake

So in modern **data lakehouses**, Parquet dominates.